COURSE NAME

DISCRETE MATHEMATICS

CSC 1204

(UNDERGRADUATE)

CHAPTER 10

TREE

M. MAHMUDUL HASAN

ASSISTANT PROFESSOR, CS, AIUB

http://www.dit.hua.gr/~m.hasan







TREE

- A tree is a connected undirected graph with no simple circuit (i.e. edge are traverse more than once)

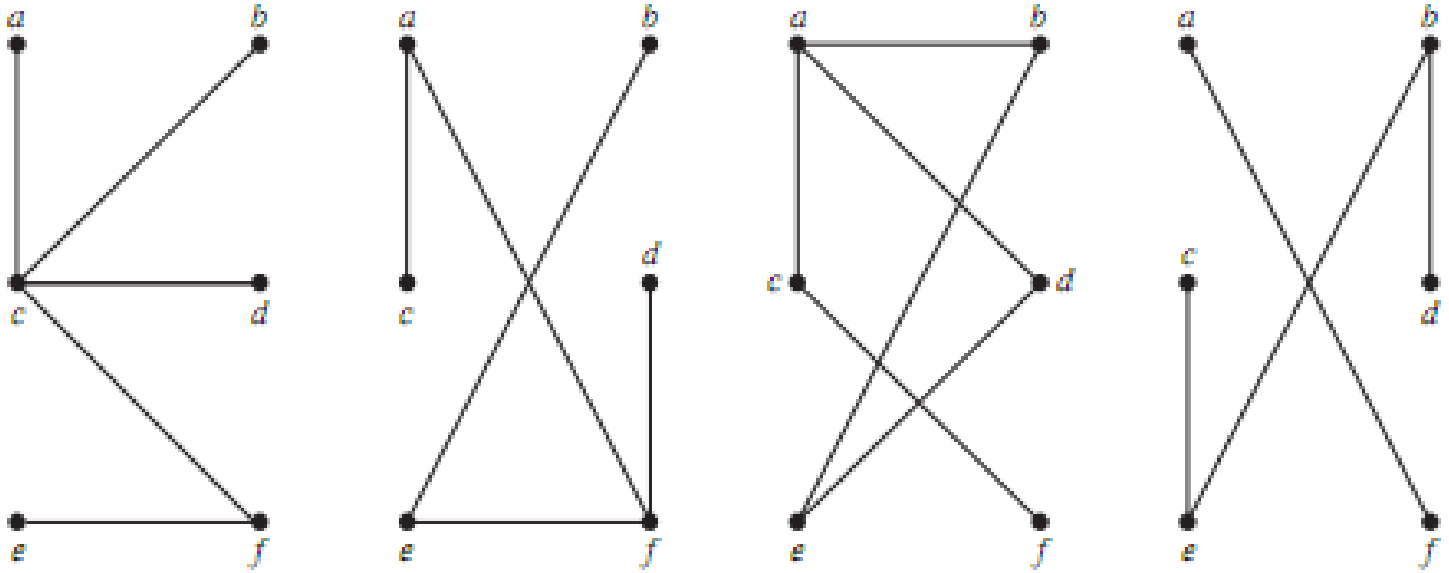


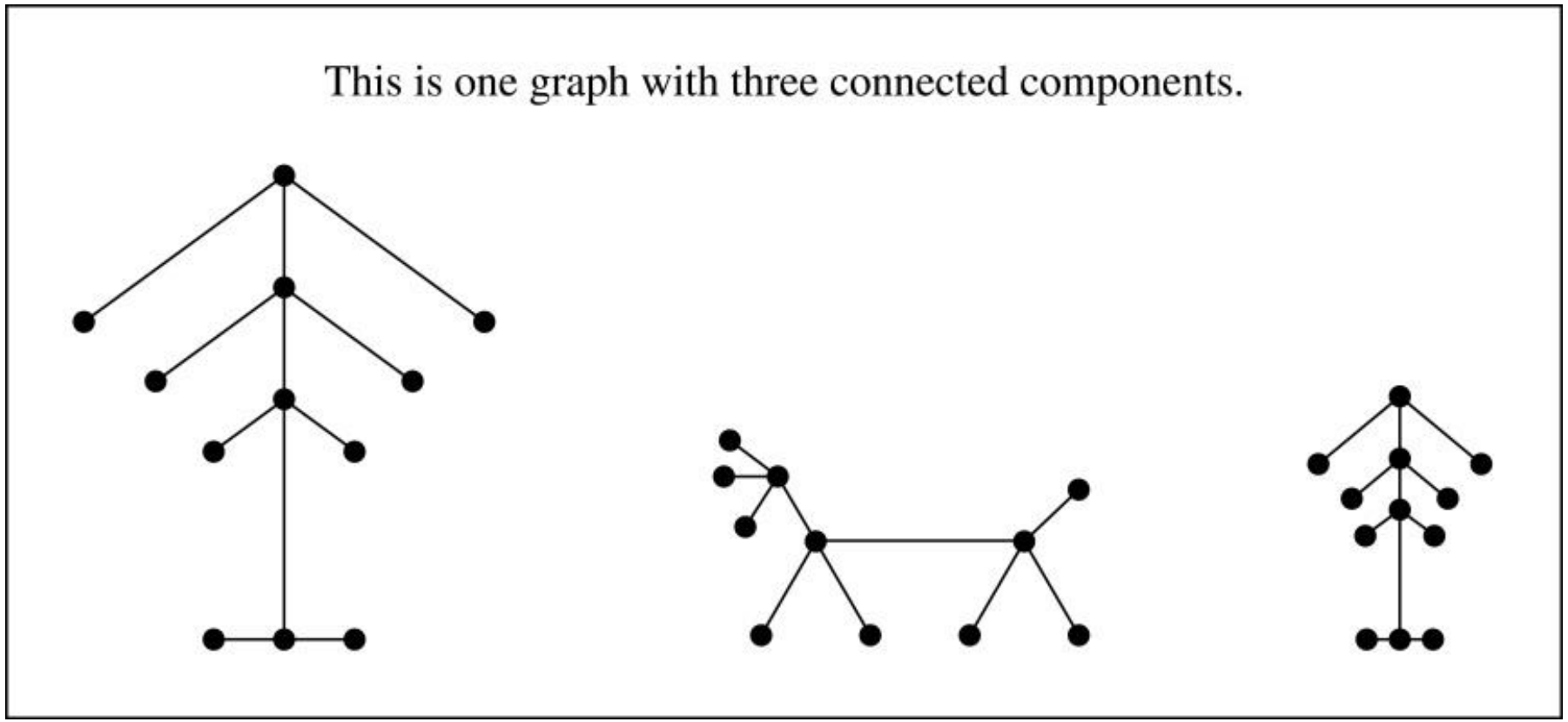
FIGURE 2 Examples of Trees and Graphs That Are Not Trees.

Solution:

- G1 and G2 are trees, because both are connected graphs with no simple circuits.
- G3 is not a tree because e, b, a, d, e is a simple circuit in this graph.
- Finally, G4 is not a tree because it is not connected.

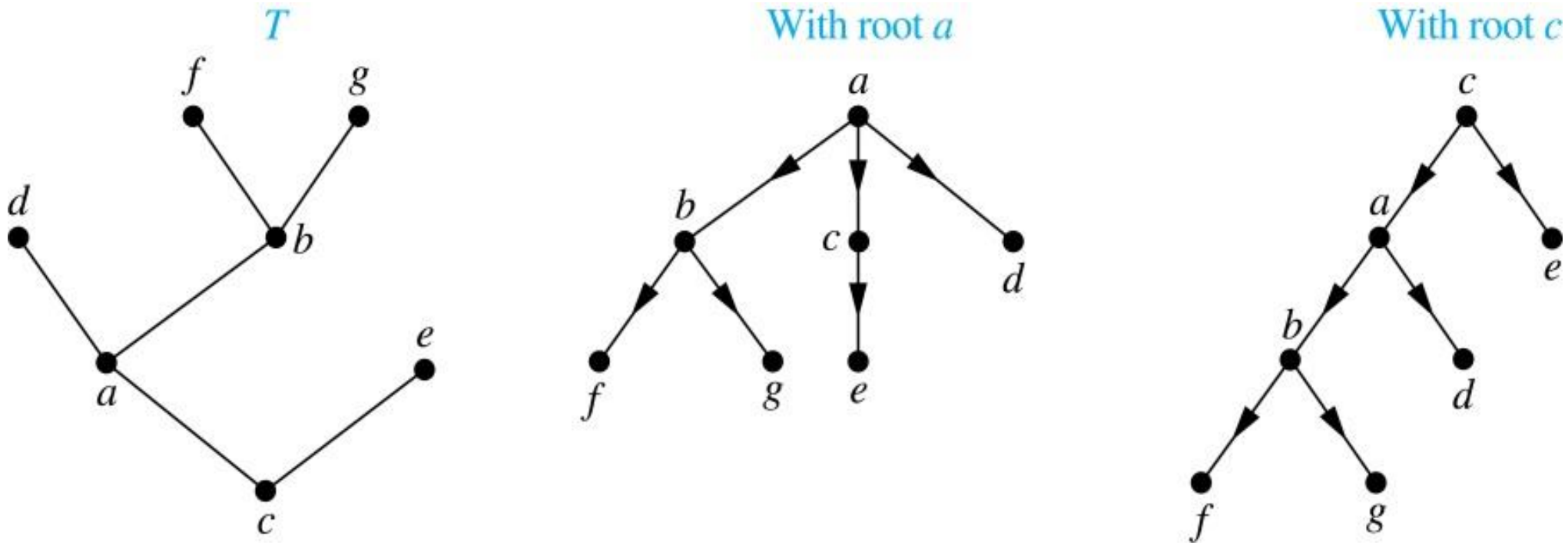
FOREST

- Theorem: An undirected graph is a **tree** if and only if there is a unique simple path between any two of its vertices.
- Forest: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree.



ROOTED TREE

- A rooted tree is a tree in which **one vertex** has been designated as the **root** and every edge is directed away from the root.
  - We can change an unrooted tree into a rooted tree by choosing any vertex as the root
  - Different choices of the root produce different rooted trees

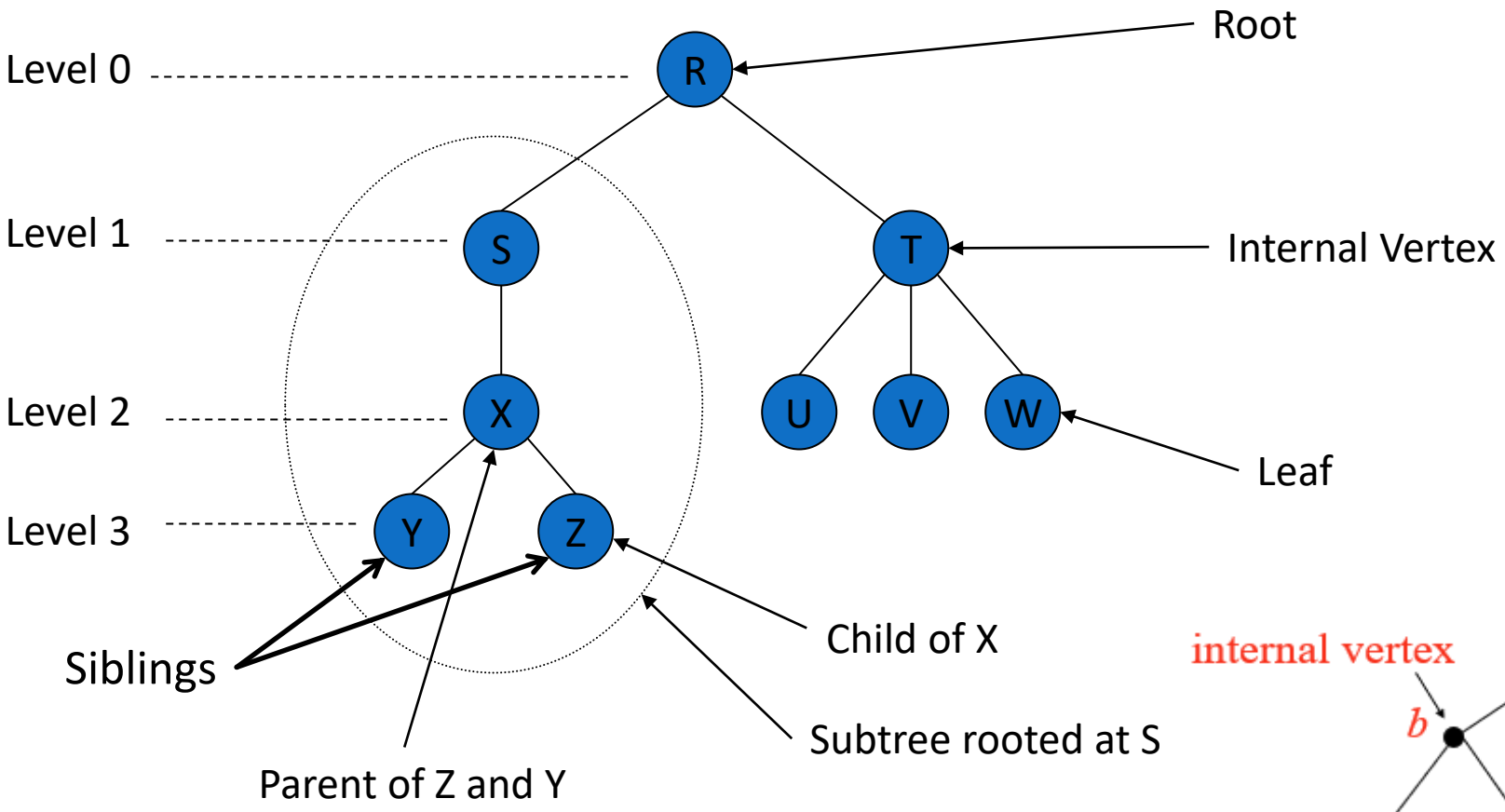


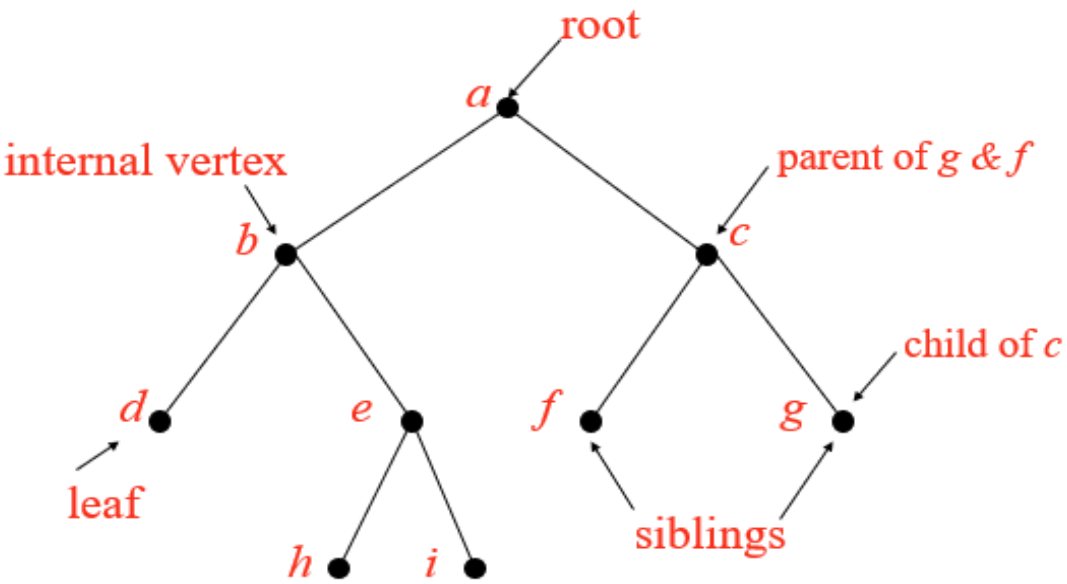
The rooted trees formed by designating ***a*** to be root and ***c*** to be the root

ROOTED TREE

- **Parent**: If *v* is a vertex of a rooted tree other than the root, the *parent* of *v* is the unique vertex *u* such that there is a directed edge from *u* to *v*. Note that such a vertex is unique
- **Child**: When *u* is a parent of *v*, *v* is called a ***child*** of *u*.
- **Siblings**: Vertices with the same parent are called ***siblings***.
- **Leaf**: A vertex of a rooted tree is called a ***leaf*** if it has no children.
- **Internal vertex:** A vertex that has children is called ***internal vertex***.
- **Note**: *The root is an internal vertex unless it is the only vertex in the graph, in which case it is a leaf.*
- **Ancestor(s):** The **ancestors** of a vertex other than the root are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root.
- **Descendant(s) :** The **descendants** of a vertex ***v*** are those vertices that have ***v*** as an ancestor.
- **Subtree**: If *a* is a vertex in a tree, the **subtree with *a* as its root** is the subgraph of the tree consisting of *a* and its descendants and all edges incident to these descendants.

ROOTED TREE


Level 0
Level 1
Level 2
Level 3
R
S
T
X
U
V
W
Y
Z
Root
Internal Vertex
Leaf
Child of X
Subtree rooted at S
Siblings
Parent of Z and Y


root
a
internal vertex
parent of g & f
b
c
child of c
d
e
f
g
leaf
siblings
h
i

ROOTED TREE

- In the rooted tree T (with root a) shown in Figure, find the ***parent*** of c, the ***children*** of g, the ***siblings*** of h, all ***internal vertices***, and all ***leaves***. What is the ***subtree*** rooted at g?

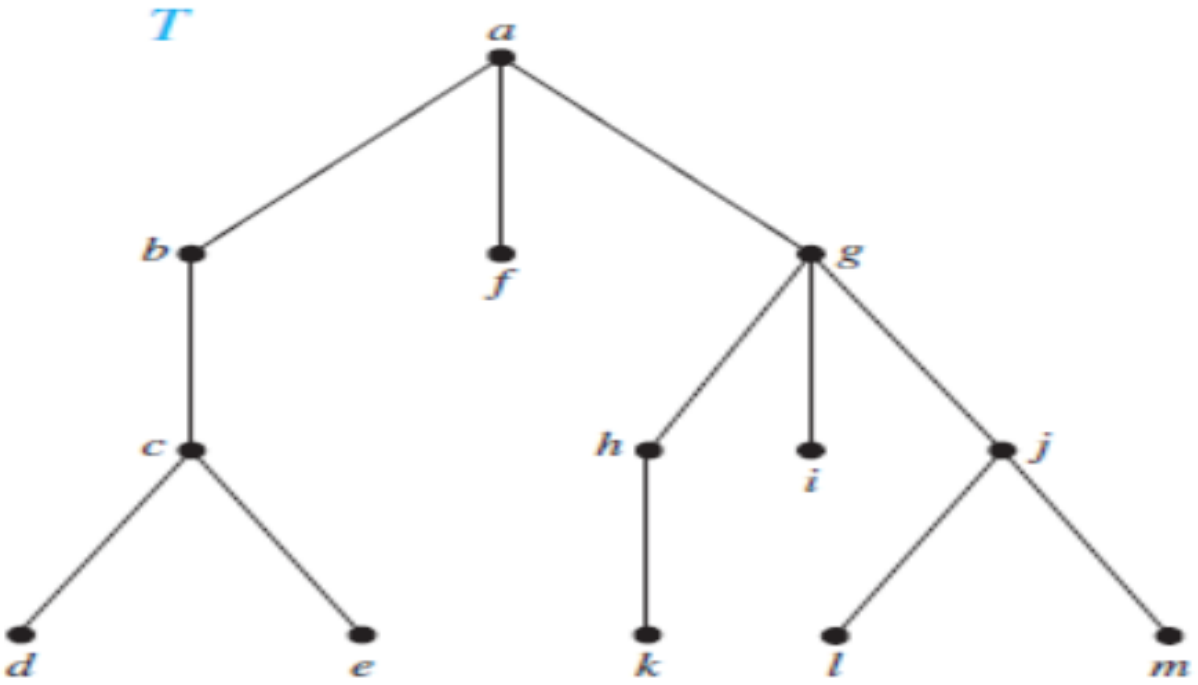


FIGURE 5 A Rooted Tree T.

Solution

- The **parent** of c is b.
- The **children** of g are h, i, and j .
- **The siblings** of h are i and j .
- **The internal vertices** are a, b, c, g, h, and j .
- The **leaves** are d, e, f , i, k, l, and m.
- The **subtree** rooted at g is shown in Figure 6(next slide)

ROOTED TREE

- In the rooted tree T (with root a) shown in Figure, find the ***parent*** of c, the ***children*** of g, the ***siblings*** of h, all ***internal vertices***, and all ***leaves***. What is the ***subtree*** rooted at g?

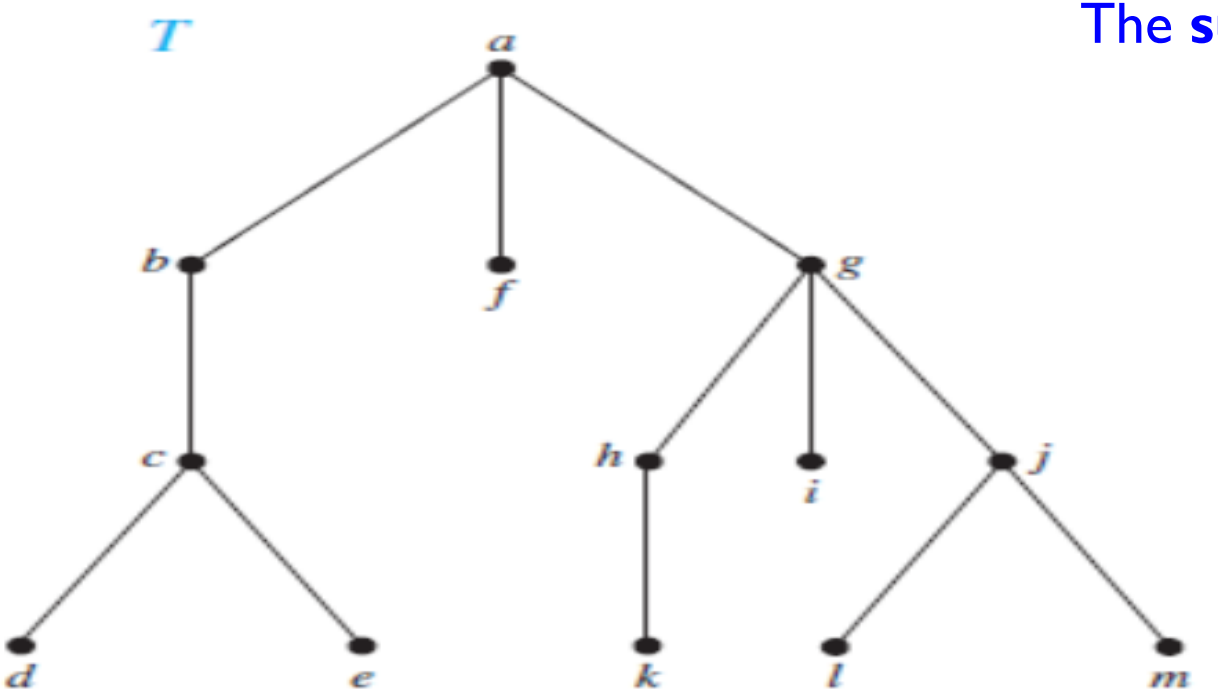


FIGURE 5 A Rooted Tree T.

The **subtree** rooted at g is shown in Figure 6

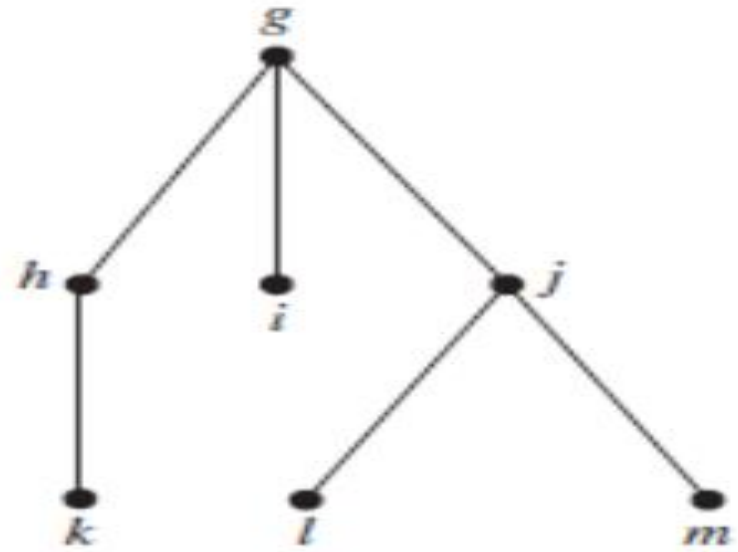


FIGURE 6 The Subtree Rooted at g.

M-ARY TREE, FULL M-ARY TREE, BINARY TREE

- A rooted tree is called an ***m-ary tree*** if every internal vertex has no more than *m* children.
- The tree is called a ***full m-ary tree*** if every internal vertex has **exactly *m* children**.
- An *m*-ary tree with ***m* = 2** is called a ***binary tree***.

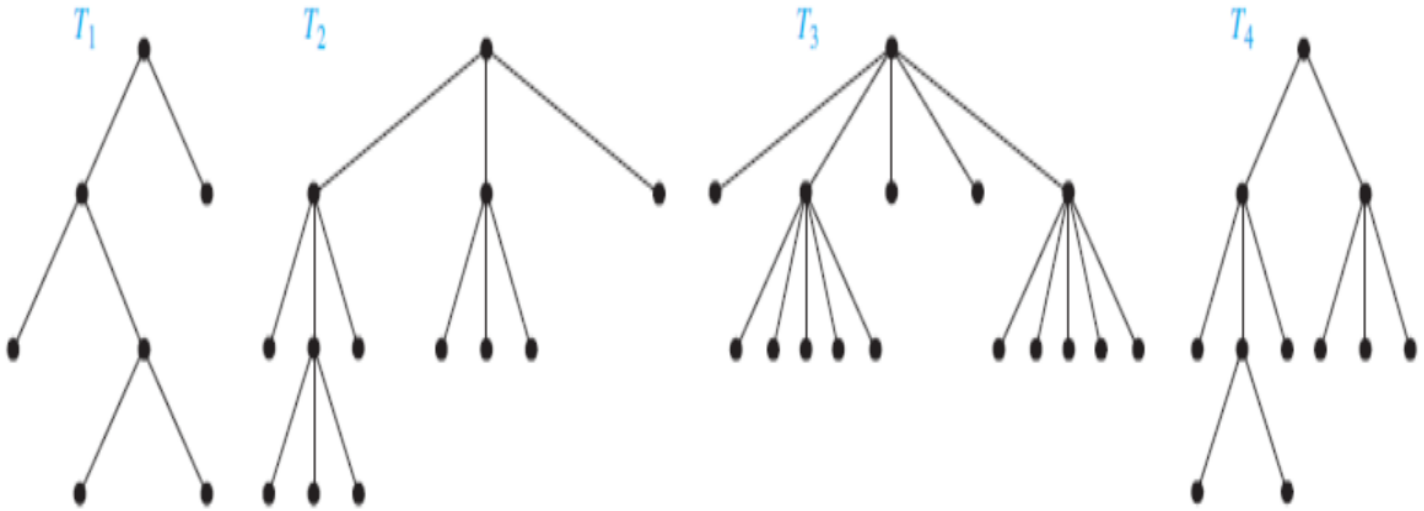


FIGURE 7 Four Rooted Trees.

- T1 is a full binary tree because each of its internal vertices has two children.
- T2 is a full 3-ary tree because each of its internal vertices has three children.
- In T3 each internal vertex has five children, so T3 is a full 5-ary tree.
- T4 is ***not*** a full m-ary tree for any *m* because some of its internal vertices have two children and others have three children.

PROPERTIES OF TREES

Q. How many edges does a tree with 10,000 vertices have?

Solution: Here, n = 10000, So, e = n –1 = 9999

Q. How many vertices does a full 5-ary tree with 100 internal vertices have?

Solution: Here, m = 5, i = 100, Therefore, n = mi + 1 = 5x100 + 1 = 501

Q. How many edges does a full binary tree with 1000 internal vertices have?

Solution:

Here, m = 2, i = 1000

So, n = mi + 1 = 2x1000 + 1 = 2001

Therefore, e = n – 1 = 2001 –1
= 2000

Q. How many leaves does a full 3-ary tree with 100 vertices have?

Solution:

Here, m = 3, n = 100

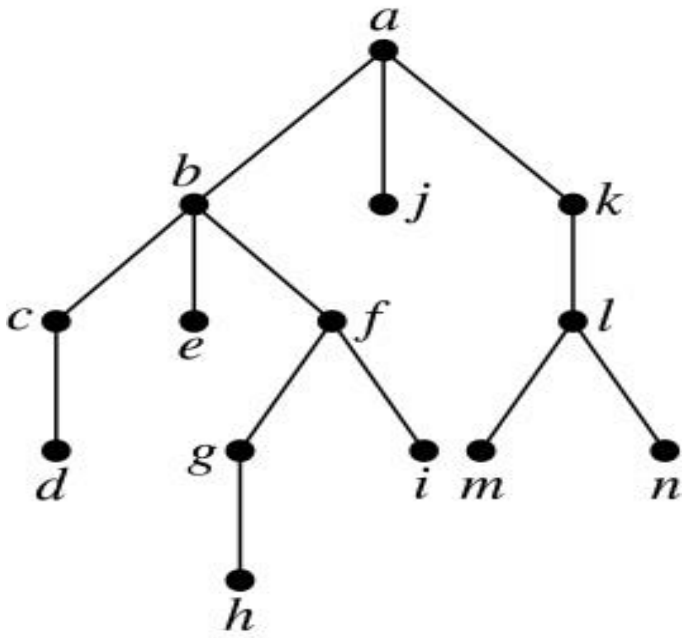
L = [(m –1)n + 1]/m

= [(3 –1)100 + 1]/ 3 = 201/3 = 67

LEVEL OF VERTICES AND HEIGHT OF TREES

- When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length. To make this idea precise we need some definitions:
- The ***level*** of a vertex *v* in a rooted tree is the **length of the unique path from the root to this vertex**.
- The ***height*** of a rooted tree is the **maximum of the levels of the vertices.**

Q. (*i*) Find the level of each vertex in the tree to the right.
(*ii*) What is the height of the tree?



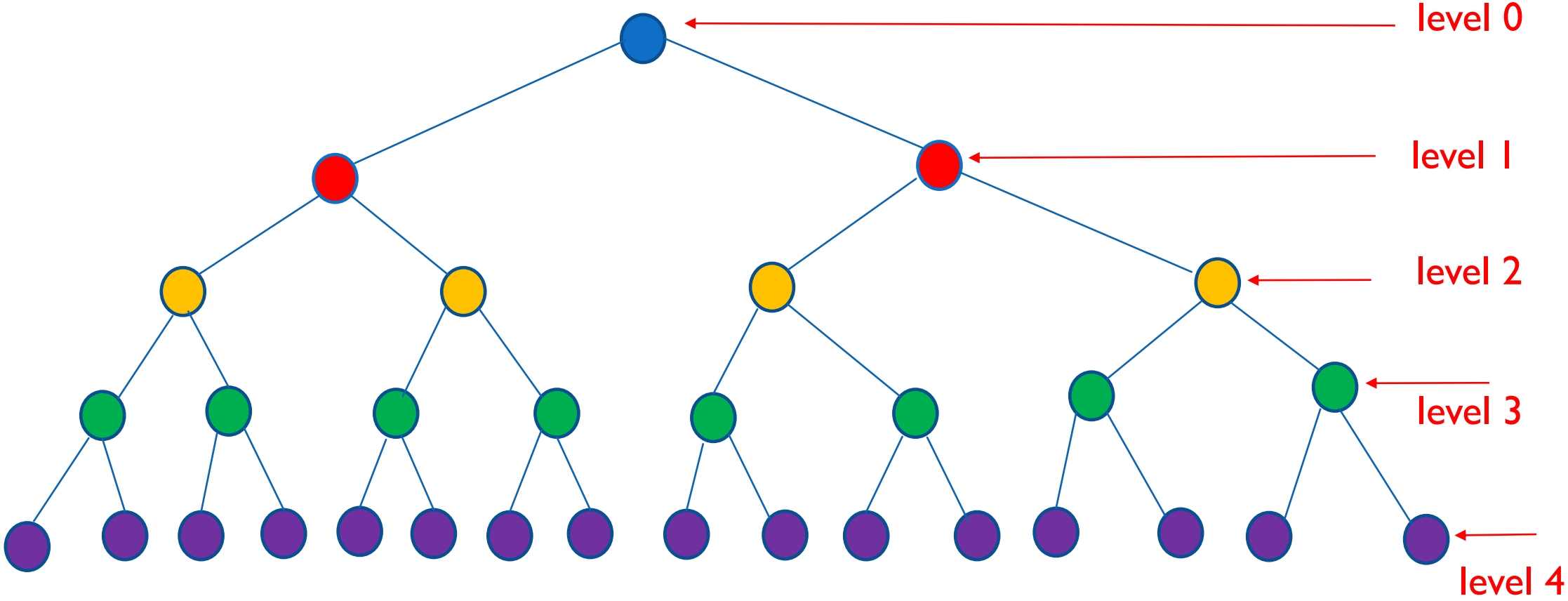
Solution:

(*i*) The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1. Vertices *c*, e, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3. Vertex *h* is at level 4.

(*ii*) The height is 4, since 4 is the largest level of any vertex.

PROPERTIES OF TREES

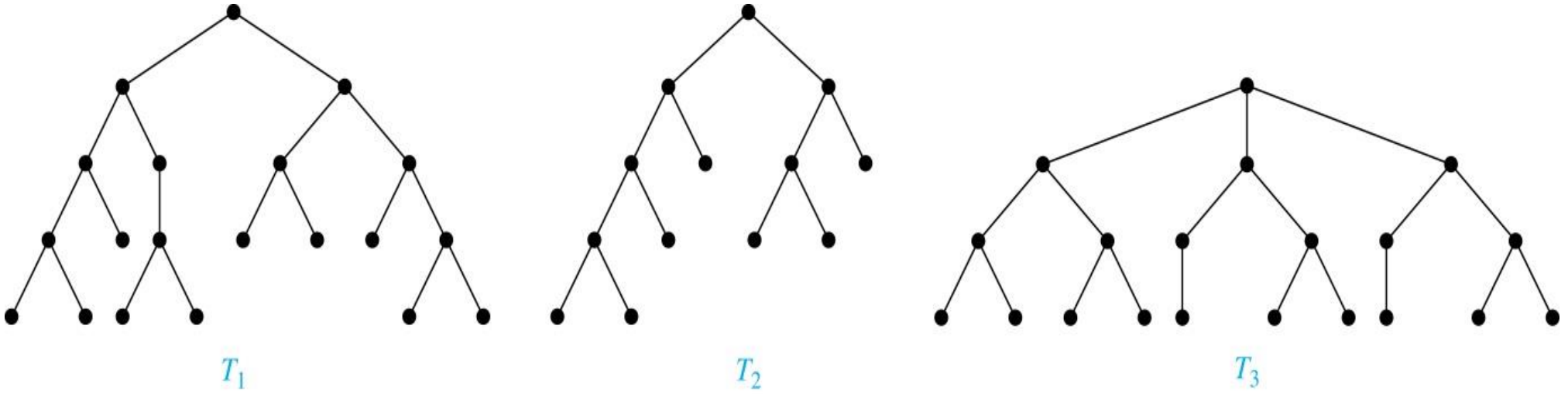
Q. Construct a complete binary tree of height 4 and a complete 3-ary tree of height 3.



BALANCED *M*-ARY TREES

Definition: A rooted *m*-ary tree of height *h* is *balanced* if all leaves are at levels *h* or *h* − 1.

Example: Which of the rooted trees shown below is balanced?



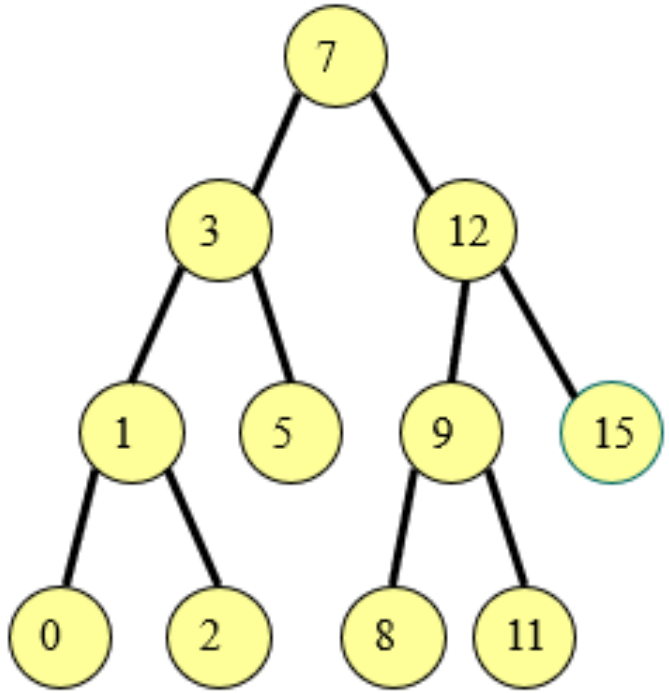
Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

Theorem: There are at most m^h leaves in an *m*-ary tree of height *h*.

BINARY SEARCH TREE (BST)

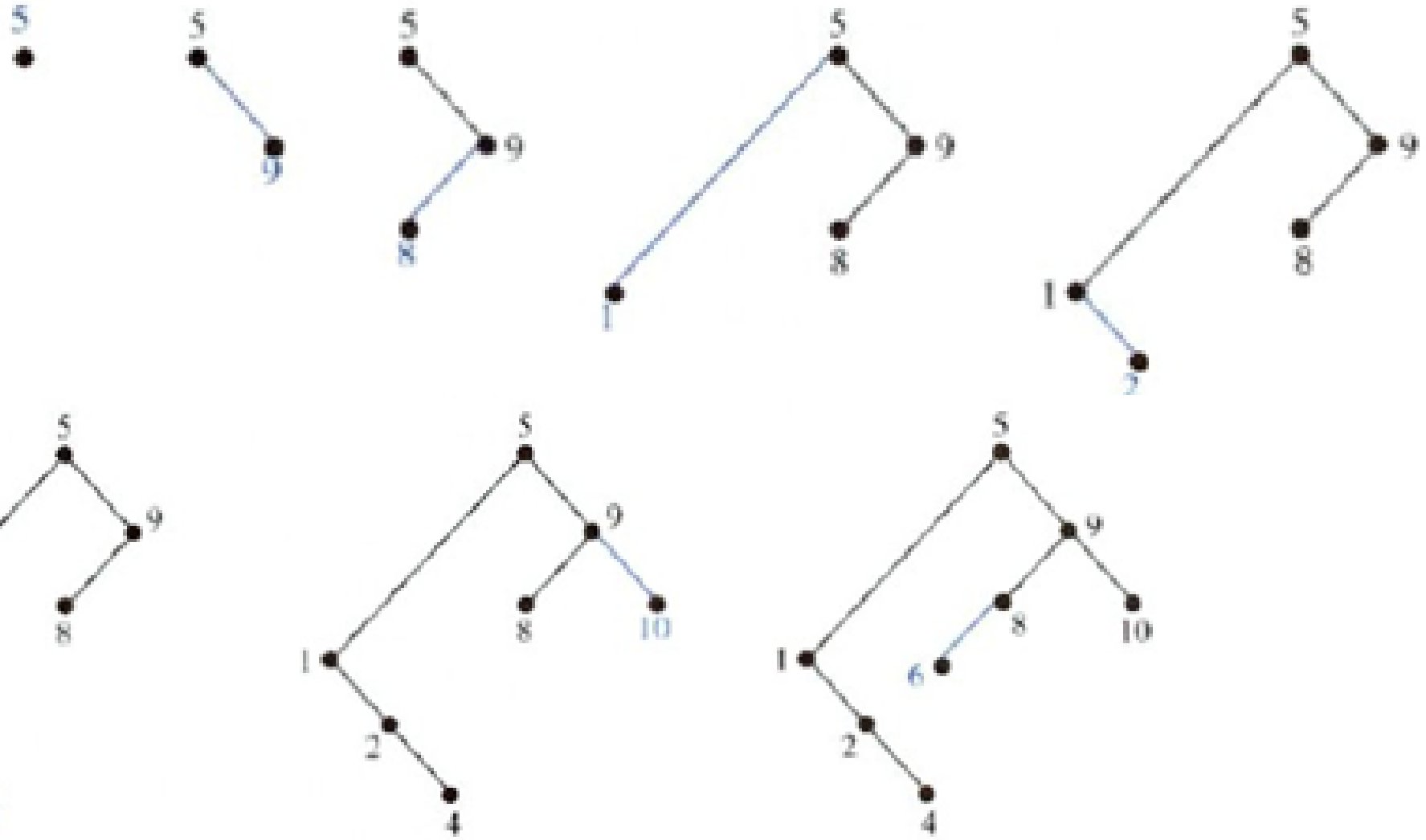
- A binary tree in which each child of a vertex is designated as a right or left child.
- No two vertex has more than one right child or left child, and each vertex is labeled with a key, which is one of the items.
- Vertices are assigned keys so that the key of a vertex is both larger than the keys of all vertices in its left subtree and smaller than the keys of all vertices in its right subtree.

- For every item *x*,
  - Every node in *x*'s left subtree is less than *x*.
  - Every node in *x*'s right subtree is greater than *x*.



BINARY SEARCH TREE (BST)

Q. Construct/Form a BST using the following numbers: 5 9 8 1 2 4 6 10



BINARY SEARCH TREE (BST)

Q. Form a binary search tree for the words mathematics, physics, geography, zoology, meteorology, geology, psychology, and chemistry (using alphabetical order).

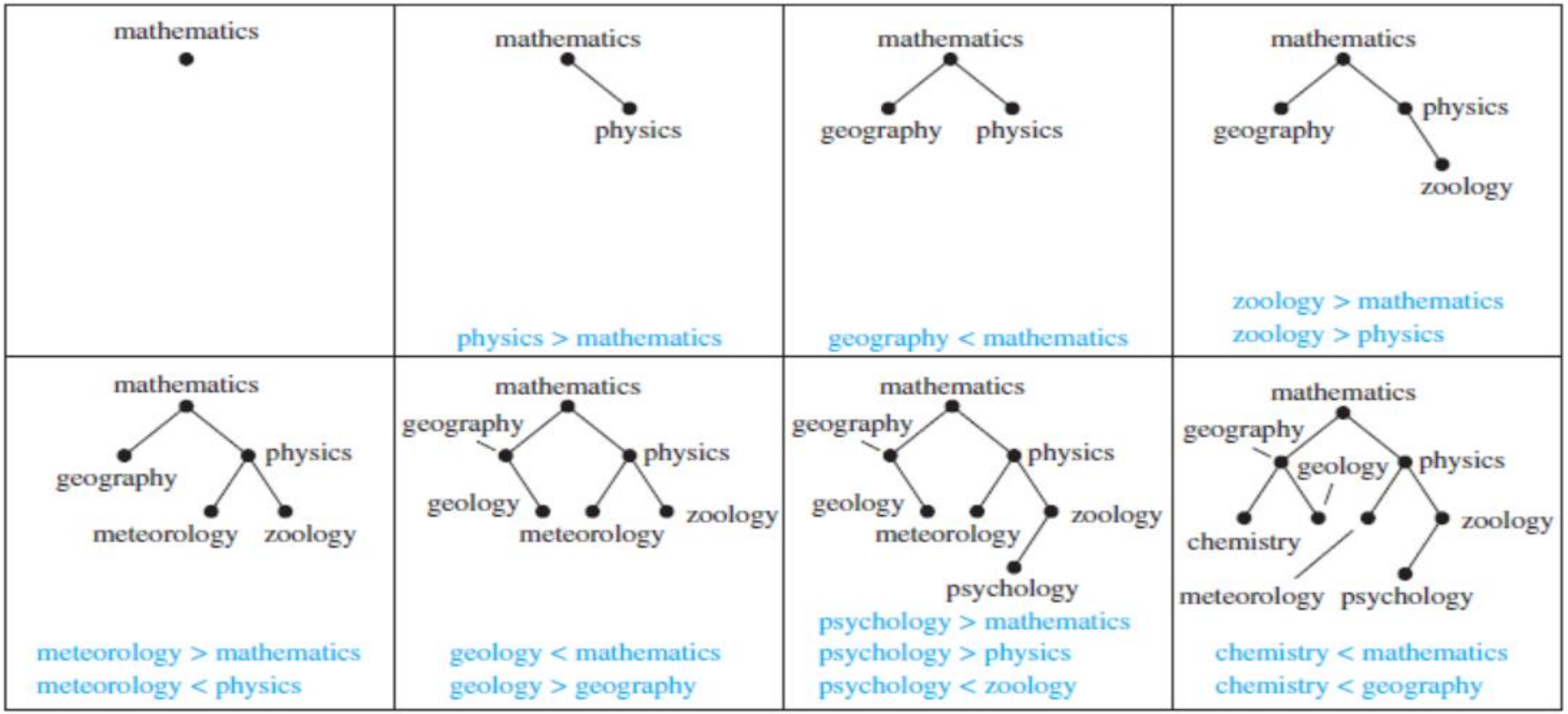


FIGURE 1 **Constructing a Binary Search Tree.**

TREE TRAVERSAL

- Tree traversal:
  - A listing of the vertices of a tree
  - Is a procedure that systematically visits every vertex of an ordered rooted tree
- Traversal Algorithms: procedures for systematically visiting every vertex of an ordered rooted tree are called traversal algorithms.
- Three most commonly used traversal algorithms:
  1) Preorder traversal
  2) Inorder traversal
  3) Postorder traversal

PREORDER TRAVERSAL

- DEFINITION: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal* of T . Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at r from left to right in T .
- The *preorder traversal* begins by visiting root *(r)*. It continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

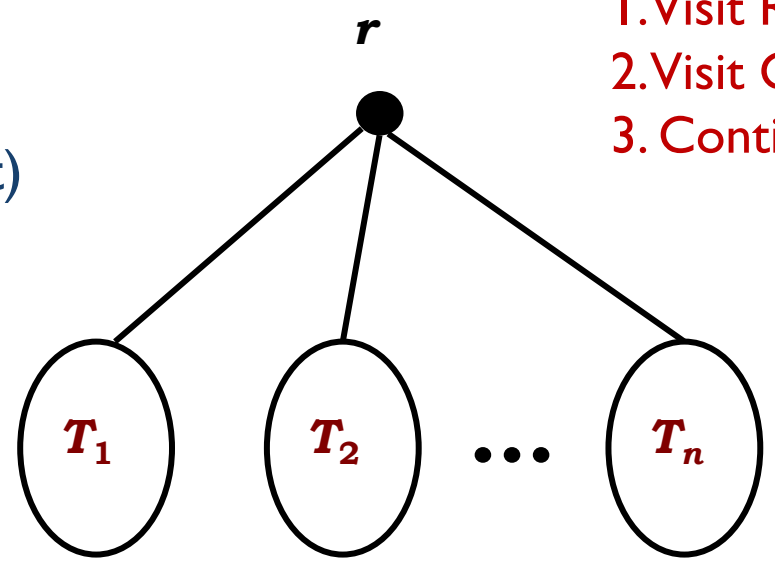
Step 1: Visit the root r

Step 2: Visit T_1 in preorder (from left to right)

Step 3: Visit T_2 in preorder

⋮

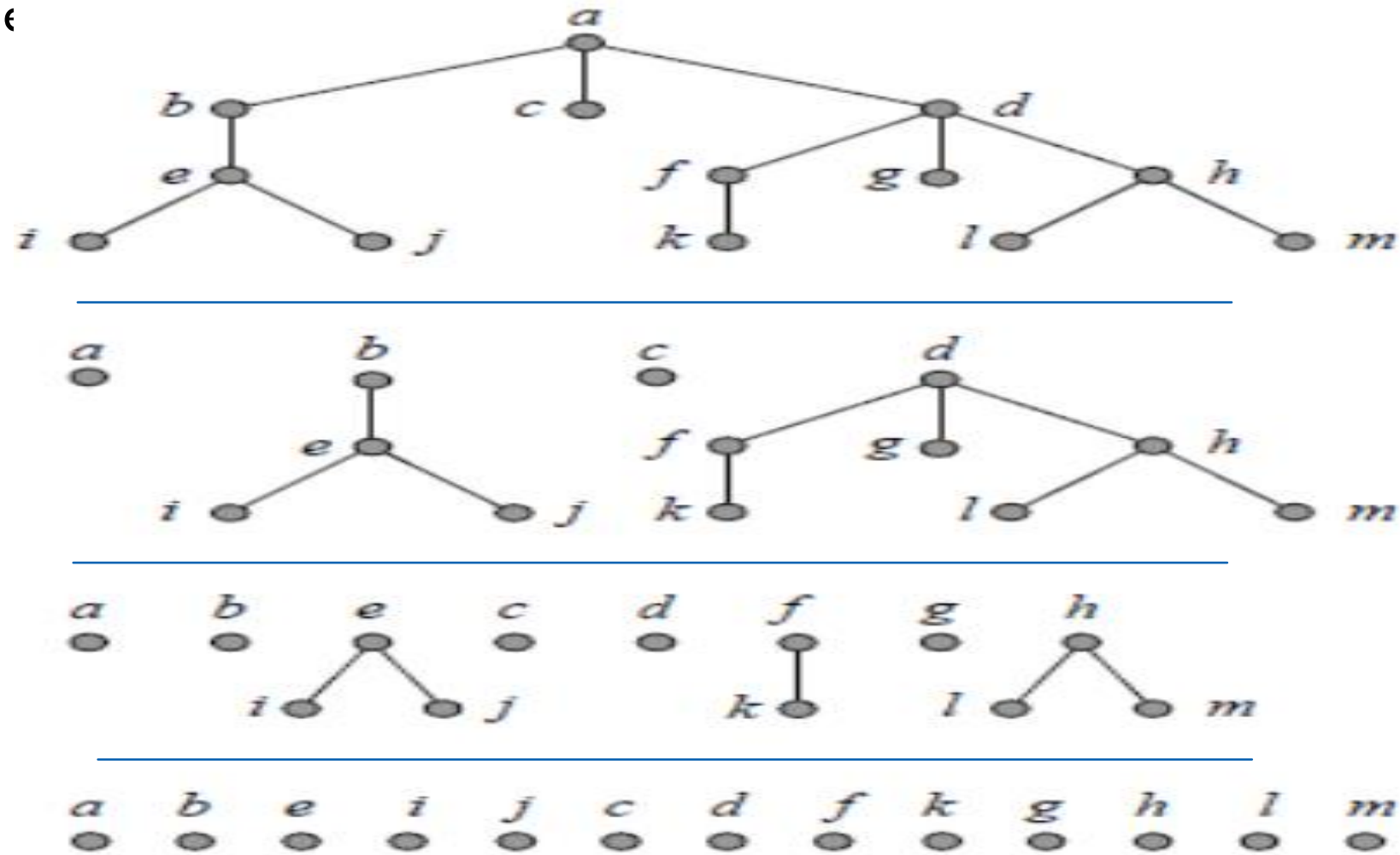
Step n+1: Visit T_n in preorder



1. Visit Root First
2. Visit Child from L→ R
3. Continue Preorder T

PREORDER TRAVERSAL

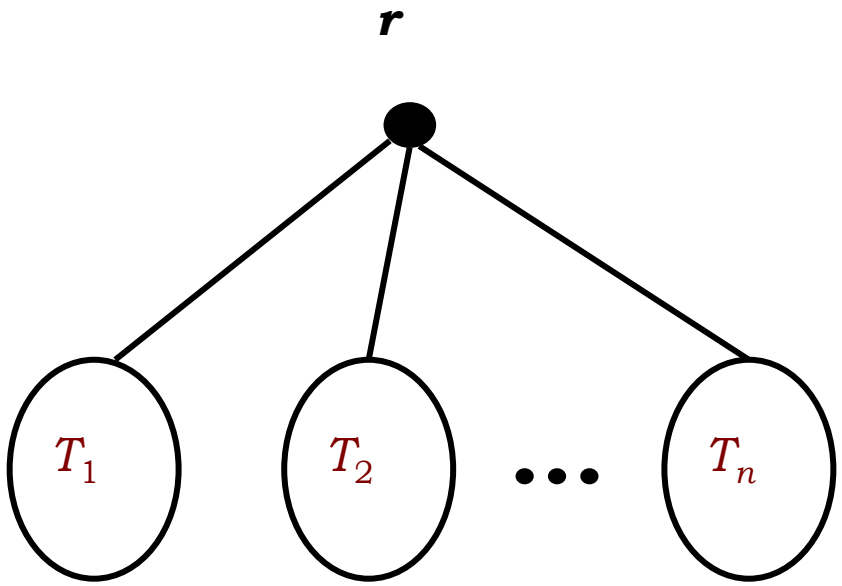
Q. In which order does a preorder traversal visit the vertices in the ordered rooted tree T shown in the



INORDER TRAVERSAL

- DEFINITION: Let T be an ordered rooted tree with root r. If T consists only of *r*, then *r* is the *inorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at *r* from left to right.
- The *inorder traversal* begins by traversing T_1 in inorder, then visiting *r*. It continues by traversing T_2 in inorder, then T_3 in inorder,..., and finally T_n in inorder.

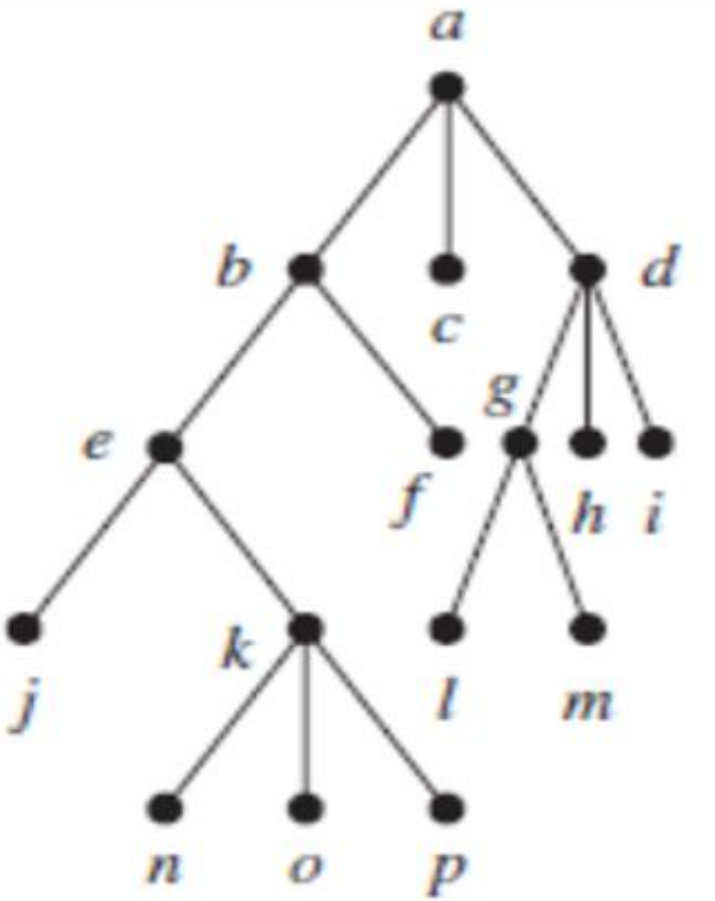
Step 1: Visit T_1 in inorder
Step 2: Visit the root *r*
Step 3: Visit T_2 in inorder
•
•
•
Step *n*+1: Visit T_n in inorder



1. Visit Child First
2. Visit Root
3. Visit Other Child L→ R

IN ORDER TRAVERSAL

Q. In which order does a inorder traversal visit the vertices in the ordered rooted tree T shown in the Figure?



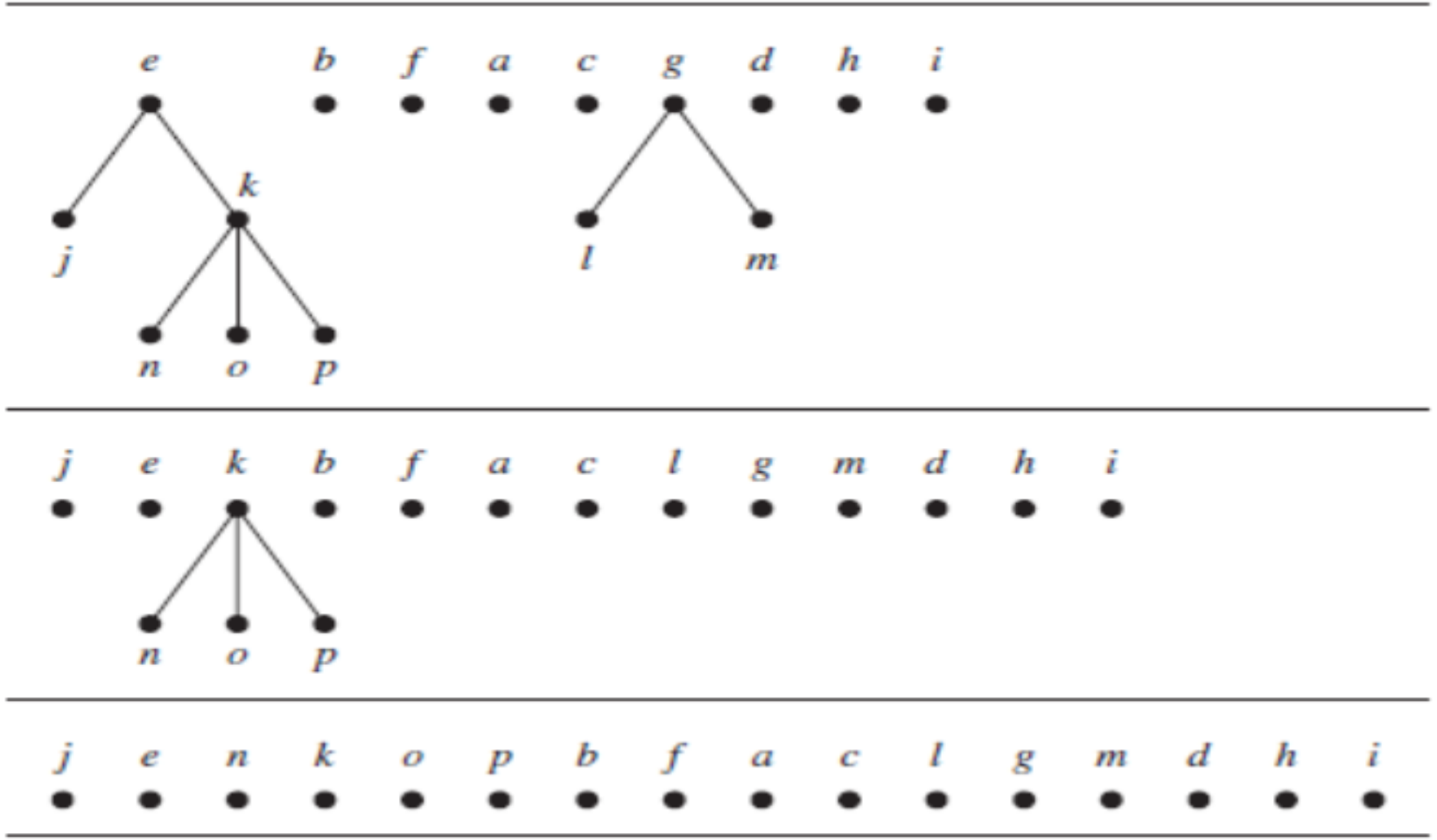


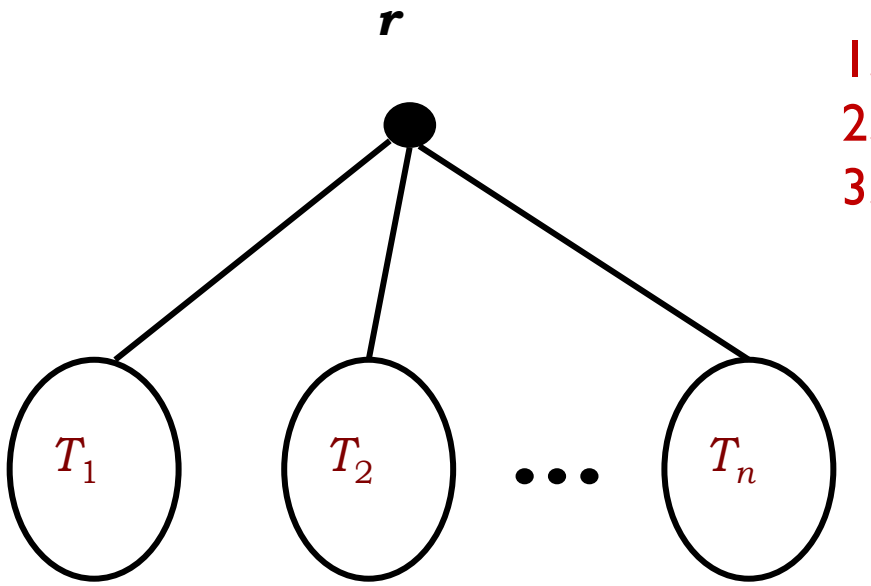
FIGURE 6 **The Inorder Traversal of T.**

POSTORDER TRAVERSAL

- DEFINITION: Let T be an ordered rooted tree with root r. If T consists only of *r*, then *r* is the post order traversal of T . Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at *r* from left to right.

  The postorder traversal begins by traversing T_1 in postorder, then T_2 in postorder, . . . , then T_n in postorder, and ends by visiting *r*.

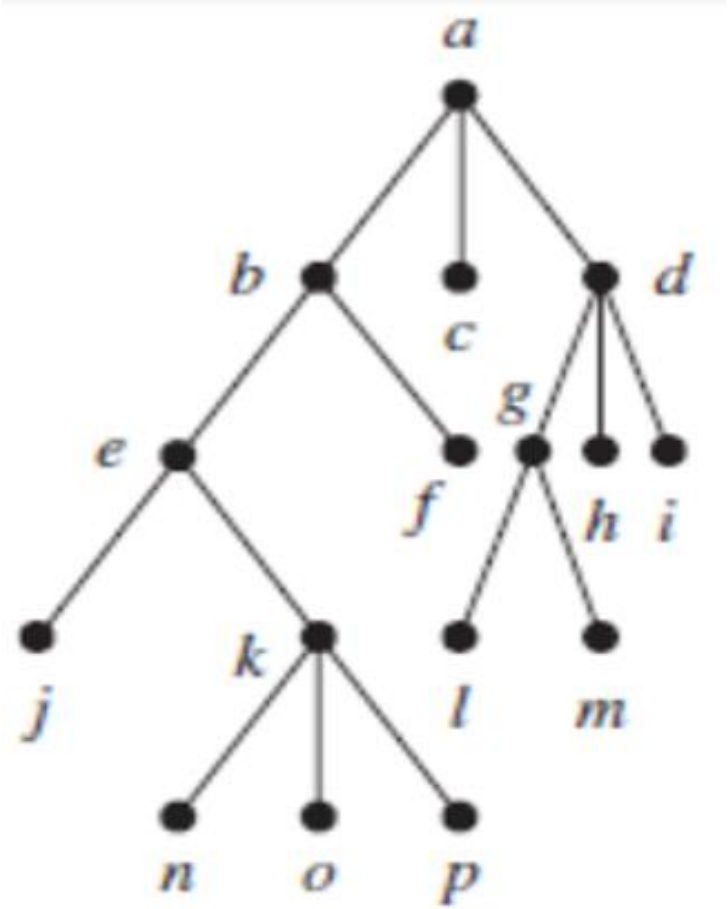
Step 1: Visit T_1 in postorder
Step 2: Visit T_2 in postorder
•
•
•
Step *n*: Visit T_n in postorder
Step *n*+1: Visit *r*



1. Visit Child First L→ R
2. Visit Root at Last
3. Continue Postorder T

POSTORDER TRAVERSAL

Q. In which order does a postorder traversal visit the vertices in the ordered rooted tree T shown in the Figure?



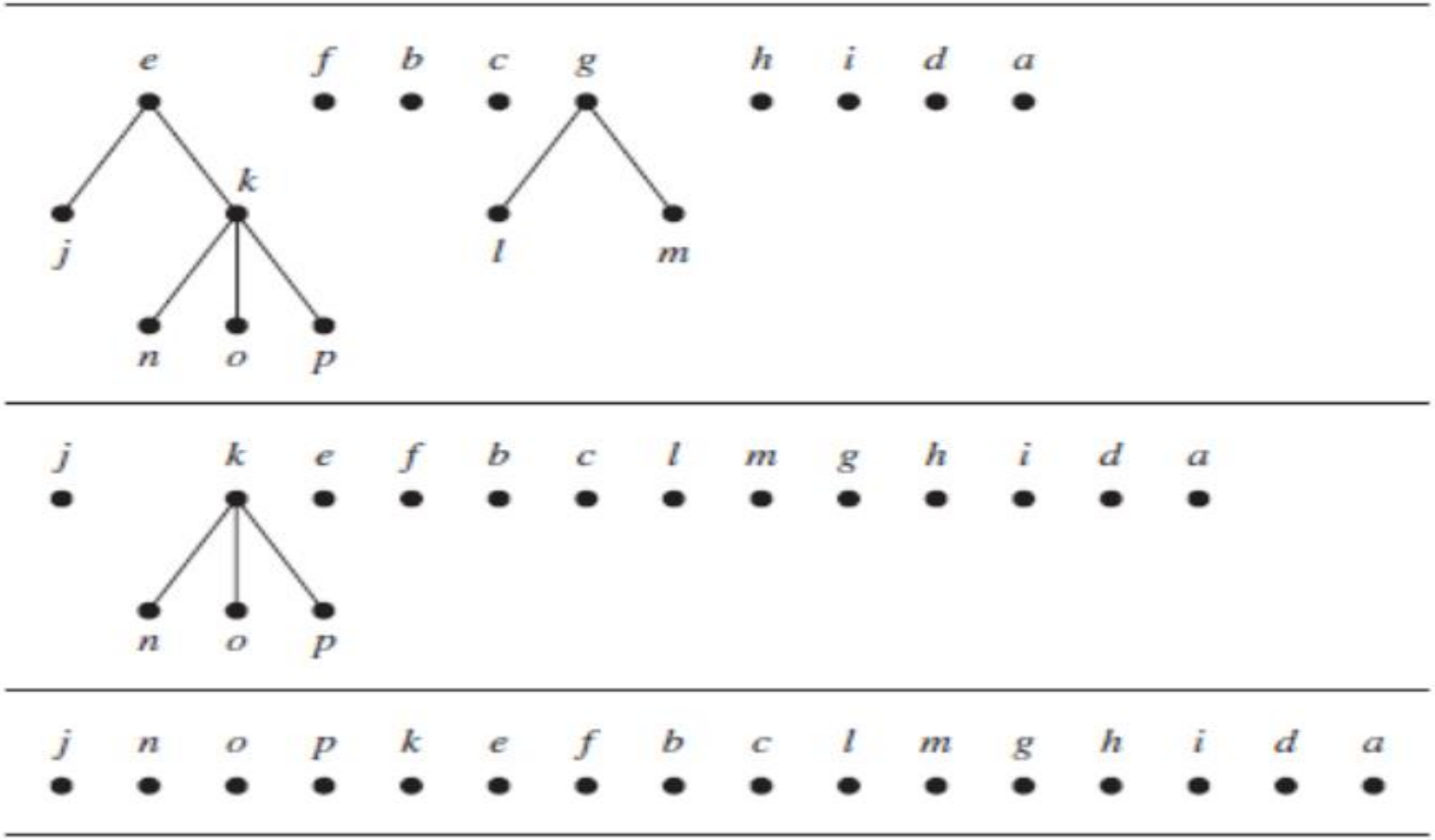


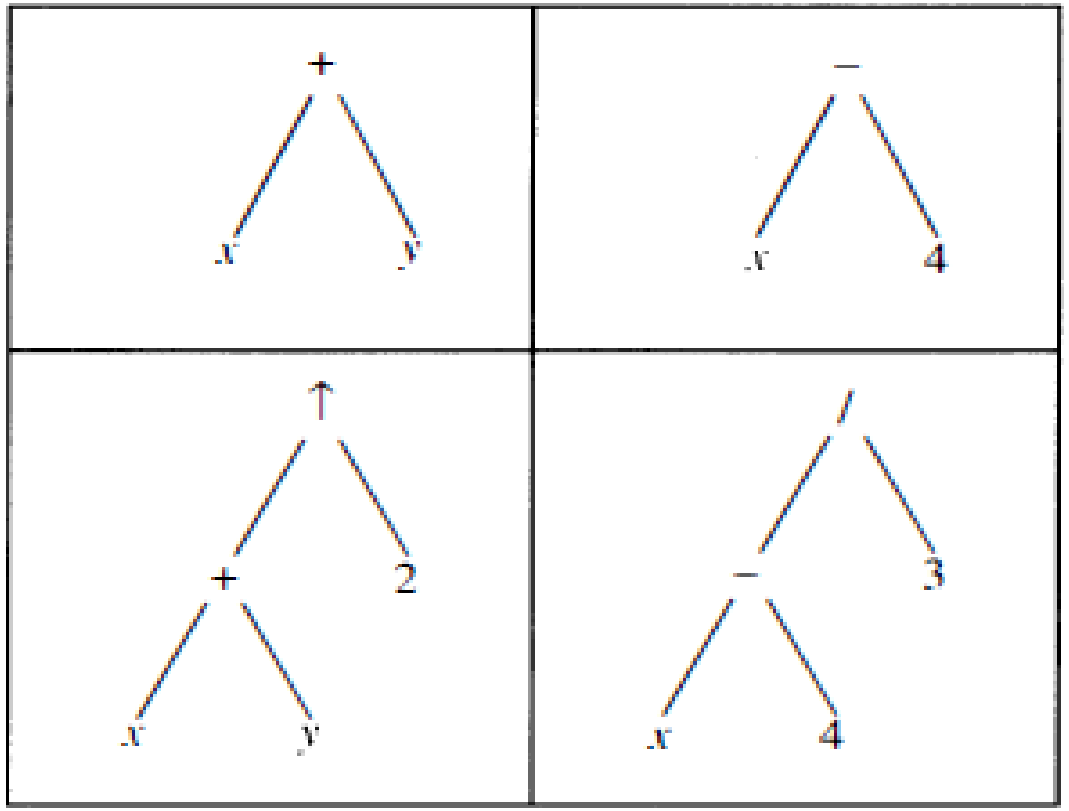
FIGURE 8 The Postorder Traversal of T.

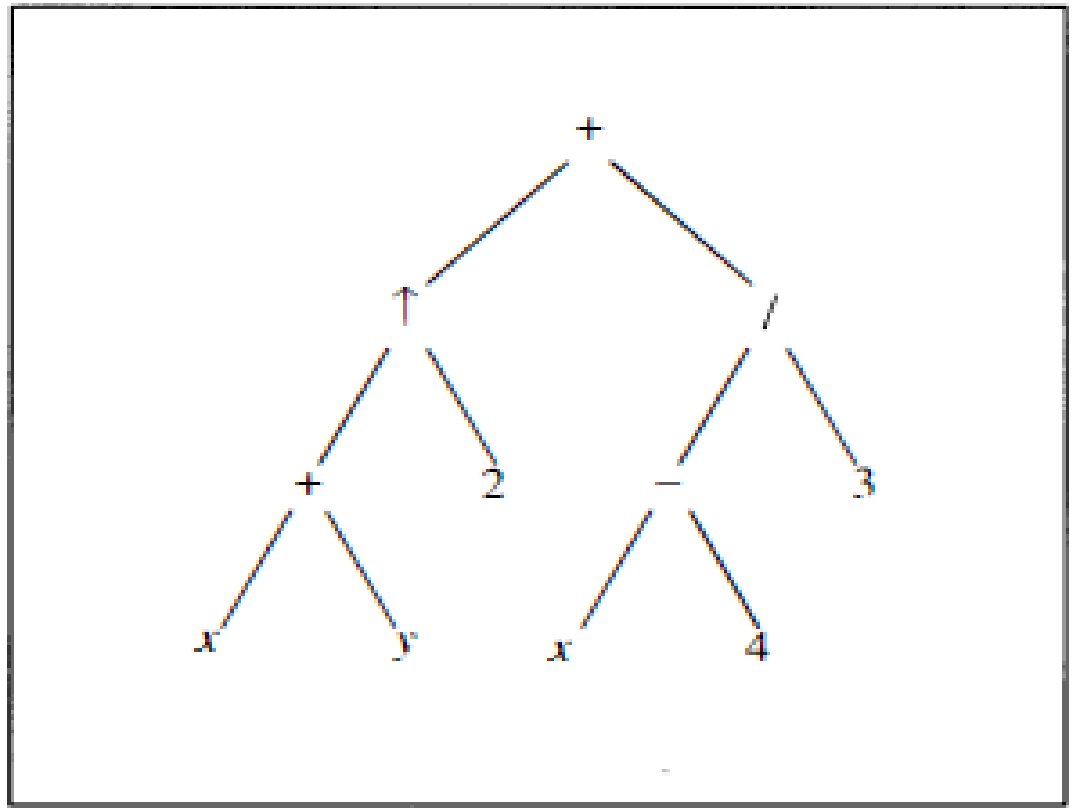
INFIX, PREFIX AND POSTFIX

- Complicated expressions, such as compound propositions, combinations of sets, and arithmetic expressions can be represented using ordered rooted tree.
- For instance, consider the representation of an arithmetic expression involving the operators + (addition), – (subtraction), * (multiplication), / (division), and ↑ (exponentiation or power).
- Complicated arithmetic expressions can be represented by an ordered rooted tree
  - **Internal vertices** represent **operators**
  - **Leaves** represent **operands**
  - **Parentheses** indicate the **order of the operations**
- Build the tree bottom-up
  - Construct smaller subtrees
  - Incorporate the smaller subtrees as part of larger subtrees

INFIX, PREFIX AND POSTFIX

Q: What is the ordered rooted tree that represents the expression ((x + y) ↑ 2) + ((x – 4) / 3) ?





Solution: Prefix: + ↑ + x y 2 / – x 4 3

Postfix: x y + 2 ↑ x 4 – 3 / +

EVALUATING A PREFIX EXPRESSION

- Prefix Evaluation
  - In a prefix expression, a binary operator precedes its two operands
  - The expression is evaluated **from right to left.** Look for the **first operator** from the right
  - Perform the corresponding operation with that operator with the two operands immediately to its right
- Postfix Evaluation
  - In a postfix expression, a binary operator follows its two operands
  - The expression is evaluated **from left to right** Look for the **first operator** from the left
  - Perform the corresponding operation with that operator with the two operands immediately to its left

Q. What is the value of the **prefix expression** **+ – * 2 3 5 / ↑ 2 3 4** ?

Q. What is the **value** of the **postfix** expression **7 2 3 * – 4 ↑ 9 3 / +** ?

EVALUATING A PREFIX EXPRESSION

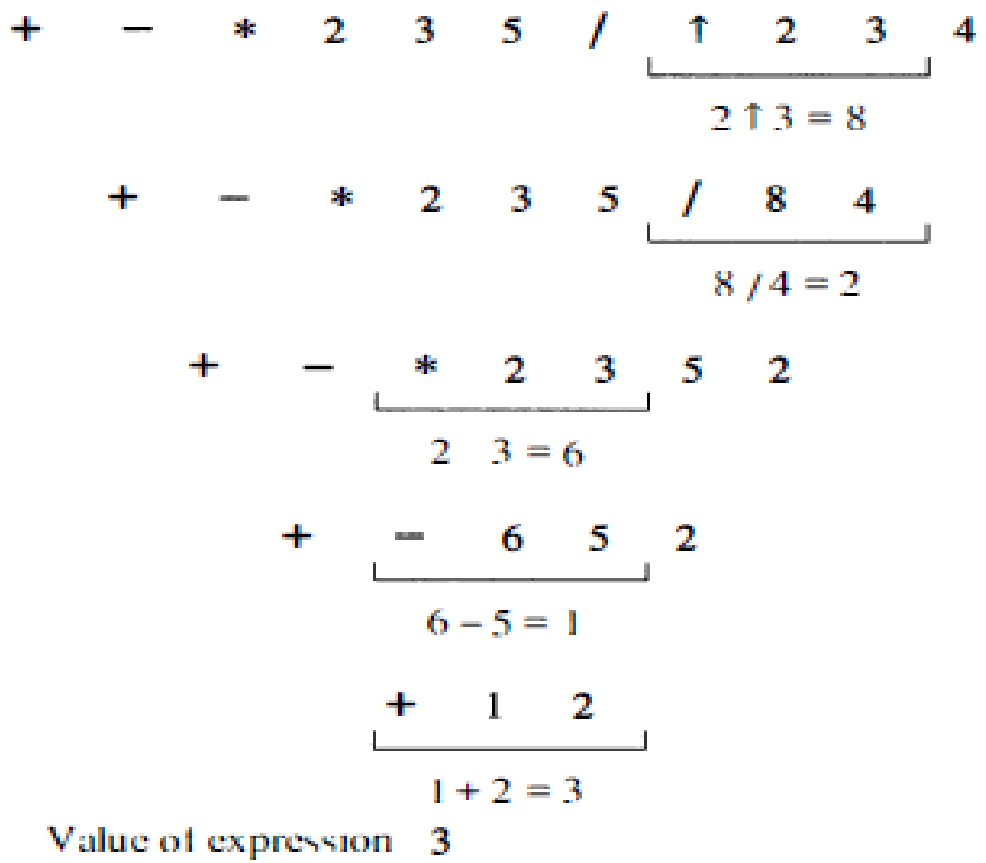


FIGURE 12 Evaluating a Prefix Expression.

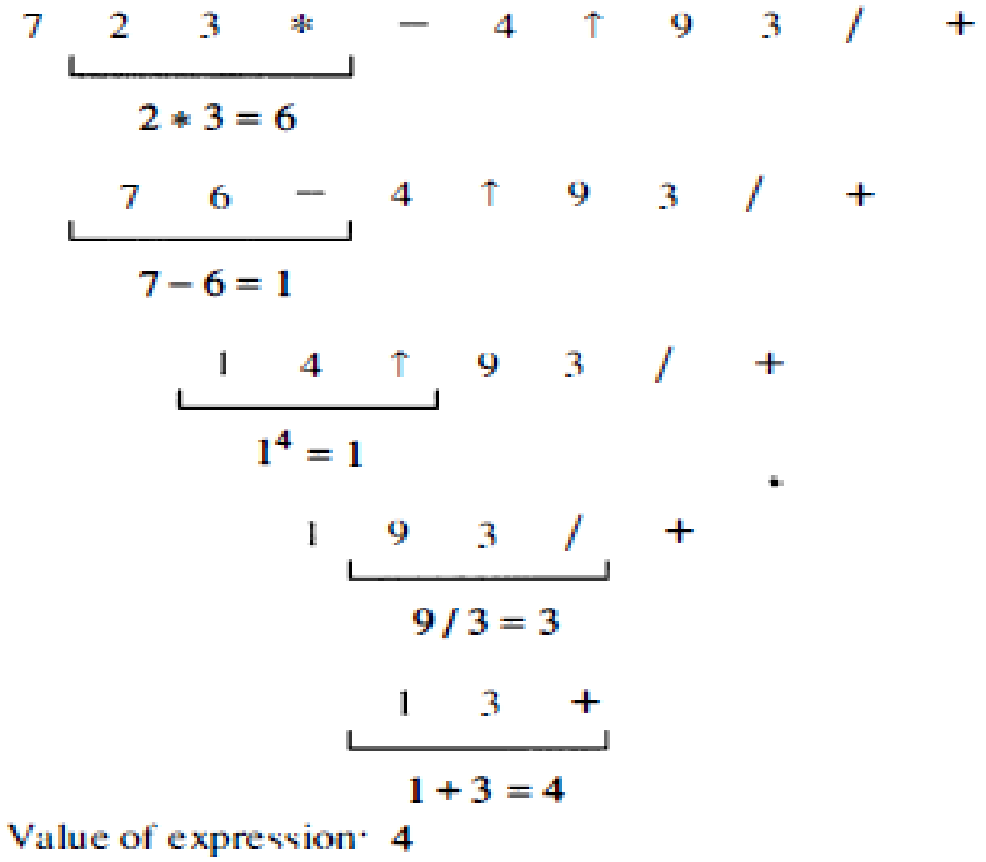


FIGURE 13 Evaluating a Postfix Expression.

REFERENCES

- *Discrete Mathematics and its applications with combinatorics and graph theory (7th edition)* by Kenneth H. Rosen [Indian Adaptation by KAMALA KRITHIVASAN], published by McGraw-Hill, is the required text.
- *SCHAUM'S ouTlines Discrete Mathematics(2nd edition)*, by Seymour Lipschutz, Marc Lipson
- *Discrete and Combinatorial Mathematics(3rd Edition)* by *R. P. Grimaldi,*, published by Addison-Wesley